PaperPilot: Document Q&A; Testbook

A larger synthetic practice PDF for testing the ShadowFox AI Engineer Intermediate workflow.

Purpose

This is an invented learning document, not an official ShadowFox document and not factual evidence about ShadowFox requirements. Use it to test PDF upload, extraction, chunking, embeddings, retrieval, and source-grounded question answering.

Suggested questions

1. What is PaperPilot? 2. Why are documents chunked? 3. What does overlap do? 4. Which embedding model is named in the example? 5. How does FAISS retrieval work in this setup? 6. What should happen when the answer is absent? 7. Why display source passages? 8. What are two limitations of PDF extraction? 9. What should never be committed to a public repository? 10. What information helps make a project reproducible?

1. Project Overview

PaperPilot is a fictional document-based question answering assistant designed for a learning exercise. A user uploads one or more PDF or TXT files, enters a question, and receives a response grounded in retrieved document passages. The application is intended to demonstrate a practical retrieval workflow rather than a general chat interface. Its main stages are ingestion, extraction, chunking, embedding, retrieval, answer generation, and source display.

Check: List the main stages in the workflow.

2. Document Ingestion

Document ingestion is the process of accepting files and preparing them for processing. The system should allow supported PDF and TXT files and should reject unsupported formats with a clear message. Each file should retain its filename so that retrieved passages can be traced back to a source. In a multi-file upload, the application should process each valid file independently and report any files that could not be read.

3. Text Extraction from PDF

PDF text extraction reads the text layer stored in a PDF. A library such as pypdf can extract text from ordinary text-based PDFs. A scanned PDF may contain page images instead of selectable text, so ordinary extraction can return little or no text. Such a file may require optical character recognition (OCR). The application should check for empty extraction results and explain the limitation instead of silently continuing.

4. Text Files and Encoding

Plain text files can be decoded using a known character encoding such as UTF-8. Some files may contain a byte-order mark or use another encoding. Robust applications should handle decoding errors safely and inform the user if the content appears empty or unreadable. Text extraction should preserve useful paragraph boundaries where possible because they help maintain context.

5. Chunking Strategy

Long documents are split into smaller chunks because embedding models and language models have input limits, and retrieval works better when passages focus on a manageable topic. A chunk size of 800 characters with 120 characters of overlap is used as an initial teaching configuration in this fictional example. Character count is not the same as token count. The best chunk size depends on the document type, the embedding model, and the questions users ask.

Check: Why might character count differ from token count?

6. Why Overlap Matters

Overlap repeats a small portion of text between adjacent chunks. It can reduce the chance that a sentence or definition near a boundary becomes separated from the surrounding explanation. Too much overlap creates redundant chunks and increases storage and retrieval work. The project should document its chosen overlap and test whether the retrieved passages contain enough context to answer questions accurately.

7. Embeddings

An embedding model maps text into a numerical vector intended to capture semantic properties. A sentence-transformers model such as all-MiniLM-L6-v2 can produce embeddings for passages and questions. The same embedding model should be used for both document chunks and queries so their vectors share the same space. Embeddings are not summaries and do not themselves produce natural-language answers.

Check: Why must query and document vectors use the same model?

8. Vector Indexing

A vector index stores document vectors so that similar vectors can be searched efficiently. FAISS provides indexes for similarity search. With normalized vectors and an inner-product index, the score corresponds to cosine similarity. The score is a relevance signal, not a guarantee that a passage answers the question. The application should inspect retrieved text and handle low-confidence or irrelevant matches.

Check: Why is a similarity score not proof of correctness?

9. Retrieval Workflow

When a user submits a question, the system embeds the query and searches the vector index for the top relevant chunks. Each result should include its source filename, chunk identifier, text, and similarity score. In a multi-document system, source metadata makes the answer easier to verify. The retrieved passages should be passed to the answer-generation stage as context.

10. Grounded Answer Generation

A grounded answer generator is instructed to use only the retrieved passages. If the passages do not contain enough evidence, it should say that the answer was not found in the uploaded documents. This policy reduces unsupported claims but does not eliminate hallucinations completely. The application should show the source passages so a user can independently check the response.

Check: What should happen when evidence is insufficient?

11. Source Visibility

Source visibility is a core requirement for document question answering. A response should identify which file supplied the supporting text and ideally display the exact retrieved passage. The interface should separate the generated answer from its evidence. A source excerpt can still be irrelevant or insufficient, so the user should be able to inspect it rather than relying only on a similarity score.

12. User Interface

A simple interface can provide a file upload component, a question input box, an answer area, and a source-context area. Gradio is one option for a lightweight Python interface. The UI should clearly show supported file types and report progress or errors. It should avoid exposing API credentials, private files, or unnecessary personal information.

13. Invalid Input Handling

The application should handle an empty question, no uploaded files, unsupported file types, unreadable PDFs, empty extracted text, and retrieval results that contain no useful evidence. Errors should be understandable and should not crash the whole session unnecessarily. Validation should occur before calling the embedding model or answer generator.

Check: Name three invalid inputs the application should handle.

14. Evaluation Questions

A basic evaluation set should include questions whose answers are explicitly present in the documents, questions that require combining two passages, and questions whose answers are absent. For present-answer questions, check whether the expected passage is retrieved and whether the generated answer is faithful. For absent-answer questions, check whether the system declines to invent an answer. Testing only easy questions can hide retrieval failures.

15. Example Evaluation Record

An evaluation record can include the question, expected answer, retrieved source filename, retrieved passage, generated answer, and a pass or fail judgment. For example, a question asking which fictional project name appears in the document should retrieve the Project Overview or Example Facts section. A question asking for a person's private phone number should not be answered unless that information is explicitly present and appropriate to disclose.

16. Security and Privacy

Uploaded documents may contain personal, confidential, or business-sensitive information. Use synthetic or redacted documents for practice. Do not commit uploaded documents to a public repository unless you have permission. API keys should be stored in an appropriate secret manager or notebook secret facility and must not be hard-coded into a notebook that will be shared publicly.

Check: What should you do before sharing uploaded files publicly?

17. Reproducibility

A reproducible project records its dependencies, model name, chunk size, overlap, retrieval top-k, and any generation settings. A requirements file helps others recreate the environment, though exact versions may need pinning after testing. The README should explain how to run the notebook or application and include screenshots of successful tests without revealing secrets.

18. Known Limitations

Text extraction may fail on scanned or malformed PDFs. Embedding similarity may retrieve a passage that shares vocabulary but does not answer the question. Small chunk sizes may lose context, while large chunks may mix unrelated topics. A prompt alone cannot guarantee factuality. These limitations should be described honestly and mitigated with validation, source display, and test cases.

Check: Name one limitation and a mitigation.

19. Improvement Ideas

Potential improvements include OCR for scanned PDFs, sentence-aware chunking, metadata filters, reranking retrieved passages, persistent vector storage, support for more document formats, and more rigorous evaluation. These improvements should be prioritized based on the assignment requirements and available time. A clear, reliable baseline is more useful than many unfinished features.

20. Summary

A practical document question answering system combines document ingestion, extraction, chunking, embeddings, vector retrieval, grounded generation, and source visibility. Each stage should be tested separately. The final application should answer from uploaded content rather than generic model memory, and it should acknowledge when the available evidence is insufficient. This fictional practice document provides content for testing those stages in a learning project.